Line wrap

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>MUM 180 — The Programme That Shows You Why You Feel This Way</title>
<meta name="description" content="You're not broken. You've been running on bad data. MUM 180 shows you the pattern — and the way out
<link rel="icon" href="data:image/svg+xml,<svg xmlns='http://www.w3.org/2000/svg' viewBox='0 0 100 100'><text y='.9em' font-size='90'
<style>
*,*::before,*::after{margin:0;padding:0;box-sizing:border-box}
:root{
  --pink:#FF69B4;
  --pink-dark:#E91E8C;
  --black:#000000;
  --gray:#333333;
  --mid:#666666;
  --light:#999999;
  --white:#FFFFFF;
  --off:#FFF5F7;
}
html{scroll-behavior:smooth}
body{font-family:system-ui,-apple-system,sans-serif;color:var(--black);background:var(--white);margin:0;line-height:1.6}

/* HEADER */
.header{background:var(--white);padding:24px 40px;text-align:center;border-bottom:2px solid #FFE4EE}
.header img{max-height:80px;width:auto}

/* HERO — THE PROBLEM */
.hero{background:var(--black);padding:100px 40px;text-align:center}
.hero-label{font-size:11px;font-weight:700;color:var(--pink);letter-spacing:0.2em;text-transform:uppercase;margin-bottom:24px}
.hero-title{font-size:clamp(32px,6vw,64px);font-weight:900;line-height:1.1;color:var(--white);max-width:800px;margin:0 auto 40px;text
.hero-title span{color:var(--pink)}
.hero-body{font-size:20px;line-height:1.7;color:rgba(255,255,255,0.6);max-width:600px;margin:0 auto 50px}
.hero-cta{display:inline-block;background:var(--pink);color:#fff;font-size:14px;font-weight:800;letter-spacing:0.1em;text-transform:u
.hero-cta:hover{background:var(--pink-dark);transform:translateY(-2px)}

/* QUOTE */
.quote{background:var(--pink);padding:60px 40px;text-align:center}
.quote p{font-size:clamp(20px,3vw,28px);font-weight:700;color:#fff;max-width:700px;margin:0 auto;line-height:1.4;text-transform:upper

/* STORYTELLING SECTION */
.story{background:var(--white);padding:100px 40px}
.story-inner{max-width:700px;margin:0 auto}
.story-block{margin-bottom:60px}
.story-label{font-size:11px;font-weight:700;color:var(--pink);letter-spacing:0.2em;text-transform:uppercase;margin-bottom:16px}
.story-title{font-size:clamp(24px,4vw,40px);font-weight:900;color:var(--black);line-height:1.2;margin-bottom:20px;text-transform:uppe
.story-body{font-size:18px;line-height:1.8;color:var(--mid)}
.story-body strong{color:var(--black)}

/* THE VILLAIN — THE LOOP */
.villain{background:var(--off);padding:100px 40px;border-top:2px solid var(--pink);border-bottom:2px solid var(--pink)}
.villain-inner{max-width:800px;margin:0 auto;text-align:center}
.villain-title{font-size:clamp(28px,4vw,48px);font-weight:900;color:var(--black);text-transform:uppercase;margin-bottom:40px;line-hei
.villain-letters{display:flex;gap:60px;justify-content:center;margin:60px 0}
.villain-char{text-align:center}
.villain-char span{display:block;font-size:100px;font-weight:900;color:var(--pink);line-height:1}
.villain-char label{font-size:14px;font-weight:700;color:var(--mid);text-transform:uppercase;letter-spacing:0.1em;margin-top:12px}
.villain-desc{font-size:18px;line-height:1.8;color:var(--mid);margin-top:50px}
.villain-desc strong{color:var(--black)}

/* THE PLAN */
.plan{background:var(--white);padding:100px 40px}
.plan-inner{max-width:800px;margin:0 auto}
.plan-title{font-size:clamp(28px,4vw,48px);font-weight:900;color:var(--black);text-transform:uppercase;margin-bottom:16px;text-align:
.plan-subtitle{font-size:18px;color:var(--mid);text-align:center;margin-bottom:60px;max-width:600px;margin-left:auto;margin-right:aut
.plan-steps{display:flex;flex-direction:column;gap:24px}
.plan-step{display:flex;gap:24px;align-items:flex-start;padding:24px;background:var(--off);border-left:4px solid var(--pink)}
.plan-num{font-size:32px;font-weight:900;color:var(--pink);line-height:1;min-width:50px}
.plan-content h3{font-size:18px;font-weight:800;color:var(--black);text-transform:uppercase;margin-bottom:8px}
.plan-content p{font-size:15px;color:var(--mid);line-height:1.6}

/* TRANSFORMATION */
.transform{background:var(--black);padding:100px 40px;text-align:center}
.transform-title{font-size:clamp(28px,4vw,48px);font-weight:900;color:#fff;text-transform:uppercase;margin-bottom:50px;line-height:1.
.transform-grid{display:grid;grid-template-columns:1fr 1fr;gap:40px;max-width:800px;margin:0 auto}
.transform-col h4{font-size:14px;font-weight:700;color:var(--light);text-transform:uppercase;letter-spacing:0.1em;margin-bottom:20px;
.transform-col.negative{opacity:0.7}
.transform-col.negative h4{color:rgba(255,255,255,0.4)}
.transform-item{font-size:16px;color:rgba(255,255,255,0.5);margin-bottom:12px;padding-left:20px;position:relative}
.transform-item::before{content:'–';position:absolute;left:0;color:var(--pink)}
.transform-item.strike{text-decoration:line-through;opacity:0.5}
.transform-item.positive{color:#fff}
.transform-item.positive::before{content:'✓'}

/* THE OFFER */
.offer{background:var(--white);padding:100px 40px;text-align:center}
.offer-inner{max-width:600px;margin:0 auto}
.offer-label{font-size:11px;font-weight:700;color:var(--pink);letter-spacing:0.2em;text-transform:uppercase;margin-bottom:16px}
.offer-title{font-size:clamp(28px,4vw,48px);font-weight:900;color:var(--black);text-transform:uppercase;margin-bottom:30px;line-heigh
.offer-features{list-style:none;text-align:left;margin:40px auto;max-width:400px}
.offer-features li{font-size:16px;color:var(--mid);padding:12px 0;border-bottom:1px solid var(--off);display:flex;align-items:center;
.offer-features li::before{content:'✓';color:var(--pink);font-weight:bold}
.offer-features li:last-child{border-bottom:none}
.offer-price{margin:40px 0}
.offer-price .old{font-size:20px;color:var(--light);text-decoration:line-through}
.offer-price .new{font-size:48px;font-weight:900;color:var(--black)}
.offer-price .period{font-size:14px;color:var(--mid)}
.offer-cta{display:inline-block;background:var(--pink);color:#fff;font-size:16px;font-weight:800;letter-spacing:0.1em;text-transform:
.offer-cta:hover{background:var(--pink-dark);transform:translateY(-2px)}
.offer-guarantee{font-size:14px;color:var(--mid);margin-top:24px}
.offer-guarantee strong{color:var(--black)}

/* FAQ */
```

```
.faq{background:var(--off);padding:80px 40px}
.faq-inner{max-width:700px;margin:0 auto}
.faq-title{font-size:clamp(24px,3vw,36px);font-weight:900;color:var(--black);text-transform:uppercase;text-align:center;margin-bottom
.faq-item{margin-bottom:24px;padding:24px;background:var(--white)}
.faq-q{font-size:16px;font-weight:700;color:var(--black);margin-bottom:12px}
.faq-a{font-size:15px;color:var(--mid);line-height:1.6}

/* CLOSING */
.closing{background:var(--pink);padding:80px 40px;text-align:center}
.closing p{font-size:clamp(24px,4vw,40px);font-weight:900;color:#fff;max-width:700px;margin:0 auto;text-transform:uppercase;line-heig
.closing .cta{display:inline-block;background:var(--black);color:#fff;font-size:14px;font-weight:800;letter-spacing:0.1em;text-transf
.closing .cta:hover{background:#333;transform:translateY(-2px)}

/* FOOTER */
.footer{background:var(--black);padding:40px;text-align:center}
.footer p{font-size:12px;color:rgba(255,255,255,0.4)}

/* RESPONSIVE */
@media(max-width:768px){
  .hero,.story,.villain,.plan,.transform,.offer,.faq,.closing{padding:60px 24px}
  .villain-letters{gap:30px}
  .villain-char span{font-size:60px}
  .transform-grid{grid-template-columns:1fr}
  .offer-cta{padding:20px 40px;font-size:14px}
}
</style>
</head>
<body>

<!-- HEADER -->
<header class="header">
<img src="logo.jpg" alt="MUM 180">
</header>

<!-- HERO — THE PROBLEM (External + Internal) -->
<section class="hero">
<div class="hero-label">This Is Not Your Fault</div>
<h1 class="hero-title">You Gave <span>Everything</span> to Your Children.</h1>
<p class="hero-body">And now you feel guilty even taking 10 minutes for yourself. You're exhausted but can't explain why. You look in
<p class="hero-body"><strong style="color:#fff">You're not broken.</strong> You've just been running on bad data.</p>
<a href="#offer" class="hero-cta">Show Me the Way Out →</a>
</section>

<!-- QUOTE — PHILOSOPHICAL PROBLEM -->
<section class="quote">
<p>"Good mums put everyone else first.<br>So why do you feel so empty?"</p>
</section>

<!-- THE VILLAIN — THE LOOP -->
<section class="villain">
<div class="villain-inner">
<h2 class="villain-title">There's a Pattern Running.<br>And It Has a Name.</h2>
<div class="villain-letters">
<div class="villain-char"><span>G</span><label>Guilty</label></div>
<div class="villain-char"><span>S</span><label>Selfish</label></div>
</div>
<p class="villain-desc">
<strong>You feel guilty BEFORE you've even done it.</strong><br>
And the whole time you're there — at the gym, getting coffee, reading this — you feel selfish.<br><br>
So you don't even get the recovery.<br>
You come back <strong>more depleted than when you left.</strong><br><br>
That loop has a name. It's called the <strong>Guilty-Selfish Loop.</strong><br>
And it's not your personality. It's a pattern.
</p>
</div>
</section>

<!-- THE STORYTELLING — WHERE IT CAME FROM -->
<section class="story">
<div class="story-inner">
<div class="story-block">
<div class="story-label">Where It Came From</div>
<h3 class="story-title">You Didn't Create This Pattern.</h3>
<p class="story-body">You inherited it. From your mother. From her mother. From the culture. From the messages that were everywhere w
</div>
<div class="story-block">
<p class="story-body"><strong>"Good mums put their children first."</strong></p>
<p class="story-body"><strong>"Nobody said motherhood would be easy."</strong></p>
<p class="story-body"><strong>"Enjoy it while it lasts."</strong></p>
</div>
<div class="story-block">
<p class="story-body">This data was <strong>WRONG.</strong></p>
<p class="story-body">Not slightly wrong. <strong style="color:var(--pink)">Fundamentally wrong.</strong> In a way that guarantees yo
<p class="story-body">Nobody told you. <strong>MUM 180 does.</strong></p>
</div>
</div>
</section>

<!-- THE PLAN -->
<section class="plan">
<div class="plan-inner">
<h2 class="plan-title">The Way Out</h2>
<p class="plan-subtitle">6 modules. 10-15 minutes each. No fluff. Just the truth.</p>
<div class="plan-steps">
<div class="plan-step">
<div class="plan-num">01</div>
<div class="plan-content">
<h3>See the Pattern</h3>
<p>Recognize the Guilty-Selfish Loop for the first time. Once you see it, it can't run invisibly anymore.</p>
</div>
</div>
<div class="plan-step">
<div class="plan-num">02</div>
<div class="plan-content">
```

```
<h3>Understand the Mechanism</h3>
<p>See exactly how the loop works — so you can catch it happening in real-time.</p>
</div>
</div>
<div class="plan-step">
<div class="plan-num">03</div>
<div class="plan-content">
<h3>Trace the Data</h3>
<p>Find where the bad data came from. Understand why it was never your fault.</p>
</div>
</div>
<div class="plan-step">
<div class="plan-num">04</div>
<div class="plan-content">
<h3>Get the Reframe</h3>
<p>The new definition of "good mum." And the words to explain it to anyone who questions it.</p>
</div>
</div>
<div class="plan-step">
<div class="plan-num">05</div>
<div class="plan-content">
<h3>Exit the Loop</h3>
<p>Interrupt the pattern. Not with willpower. With knowledge. Once you know, you can choose.</p>
</div>
</div>
<div class="plan-step">
<div class="plan-num">06</div>
<div class="plan-content">
<h3>Return to Yourself</h3>
<p>Come home. Not to who you were. To who you were always supposed to be.</p>
</div>
</div>
</div>
</div>
</section>

<!-- TRANSFORMATION -->
<section class="transform">
<h2 class="transform-title">Before & After</h2>
<div class="transform-grid">
<div class="transform-col negative">
<h4>Before</h4>
<p class="transform-item strike">Feeling guilty before doing anything for myself</p>
<p class="transform-item strike">Exhausted but can't explain why</p>
<p class="transform-item strike">Looking in the mirror and not recognizing myself</p>
<p class="transform-item strike">Thinking something is wrong with me</p>
<p class="transform-item strike">Running on empty, no refill</p>
</div>
<div class="transform-col positive">
<h4>After</h4>
<p class="transform-item positive">Knowing why I feel this way</p>
<p class="transform-item positive">Having the language for the pattern</p>
<p class="transform-item positive">Taking action without the guilt</p>
<p class="transform-item positive">Being a whole person — not just a mum</p>
<p class="transform-item positive">Coming home to myself</p>
</div>
</div>
</section>

<!-- THE OFFER -->
<section class="offer" id="offer">
<div class="offer-inner">
<div class="offer-label">MUM 180 — Complete Programme</div>
<h2 class="offer-title">Everything You Need.</h2>
<ul class="offer-features">
<li>6 modules (10-15 min each)</li>
<li>Lifetime access to all content</li>
<li>The Guilty-Selfish Loop framework</li>
<li>Private community access</li>
<li>The new definition of "good mum"</li>
<li>Words to explain it to anyone</li>
<li>14-day money-back guarantee</li>
</ul>
<div class="offer-price">
<div class="old">$97</div>
<div class="new">$47</div>
<div class="period">one-time payment • lifetime access</div>
</div>
<a href="access.html" class="offer-cta">Join Now — $47</a>
<p class="offer-guarantee"><strong>100% Money-Back Guarantee:</strong> If it doesn't resonate within 14 days, email us for a full ref
</div>
</section>

<!-- FAQ -->
<section class="faq">
<div class="faq-inner">
<h2 class="faq-title">Questions?</h2>
<div class="faq-item">
<p class="faq-q">I'm already in therapy. Is this for me?</p>
<p class="faq-a">Absolutely. MUM 180 works alongside other support. Think of it as additional context — not a replacement.</p>
</div>
<div class="faq-item">
<p class="faq-q">How long do the modules take?</p>
<p class="faq-a">About 10-15 minutes each. You can do them all in one sitting or spread them out over a week.</p>
</div>
<div class="faq-item">
<p class="faq-q">What if I don't have time?</p>
<p class="faq-a">That's exactly who this is for. Especially if you don't have time. (You'll understand when you read Module 1.)</p>
</div>
<div class="faq-item">
<p class="faq-q">Is this religious?</p>
<p class="faq-a">No. MUM 180 is secular and science-based. It's for any mum, regardless of background.</p>
</div>
<div class="faq-item">
```

```
<p class="faq-q">What if it doesn't help?</p>
<p class="faq-a">14-day money-back guarantee. If it doesn't resonate, email us for a full refund. No questions asked.</p>
</div>
</div>
</section>

<!-- CLOSING -->
<section class="closing">
<p>You're not lazy. You're not broken.<br>You've just been running on bad data.<br>Now you can do something about it.</p>
<a href="#offer" class="cta">Start MUM 180 — $47 →</a>
</section>

<!-- FOOTER -->
<footer class="footer">
<p>© 2025 MUM 180. All rights reserved.</p>
</footer>


<script>
/**
 * Iframe 元素高亮注入脚本
 * 需要在目标网站中引入此脚本来支持跨域 iframe 高亮功能
 *
 * 使用方法:
 * 1. 将此脚本添加到目标网站的 HTML 中
 * 2. 或通过浏览器扩展、用户脚本等方式注入
 */

(function () {
  "use strict";

  // 检查是否在 iframe 中
  if (window.self === window.top) {
    return; // 不在 iframe 中，不执行
  }

  // 检查是否已经初始化过
  if (window.__iframeHighlightInitialized) {
    return;
  }
  window.__iframeHighlightInitialized = true;
  console.log("Iframe 高亮脚本已加载");

  // 创建高亮覆盖层
  var overlay = document.createElement("div");
  overlay.id = "iframe-highlight-overlay";
  overlay.style.cssText = "\n    position: fixed;\n    top: 0;\n    left: 0;\n    width: 100vw;\n    height: 100vh;\n    pointer-even

  // 创建悬停高亮框（虚线边框）
  var highlightBox = document.createElement("div");
  highlightBox.id = "iframe-highlight-box";
  highlightBox.style.cssText = "\n    position: absolute;\n    border: 2px dashed #007AFF;\n    background: rgba(0, 122, 255, 0.08);\

  // 创建选中节点的常驻高亮框（实线边框）
  var selectedBox = document.createElement("div");
  selectedBox.id = "iframe-selected-box";
  selectedBox.style.cssText = "\n    position: absolute;\n    border: 2px solid #007AFF;\n    pointer-events: none;\n    display: non

  // 创建悬停标签显示
  var tagLabel = document.createElement("div");
  tagLabel.id = "iframe-tag-label";
  tagLabel.style.cssText = "\n    position: absolute;\n    background: #007AFF;\n    color: white;\n    padding: 2px 6px;\n    font-s

  // 创建选中节点标签
  var selectedLabel = document.createElement("div");
  selectedLabel.id = "iframe-selected-label";
  selectedLabel.style.cssText = "\n    position: absolute;\n    background: #007AFF;\n    color: white;\n    padding: 3px 8px;\n    f
  overlay.appendChild(highlightBox);
  overlay.appendChild(selectedBox);
  overlay.appendChild(tagLabel);
  overlay.appendChild(selectedLabel);
  document.body.appendChild(overlay);

  // 存储当前选中的元素
  var selectedElement = null;
  var highlightEnabled = false;

  // 更新选中元素的高亮显示
  function updateSelectedHighlight(element) {
    console.log("updateSelectedHighlight called with:", element);
    if (!element) {
      selectedBox.style.display = "none";
      selectedLabel.style.display = "none";
      selectedElement = null;
      console.log("Cleared selected highlight");
      return;
    }
    selectedElement = element;
    var rect = element.getBoundingClientRect();
    console.log("Selected element rect:", rect);

    // 更新选中高亮框位置
    selectedBox.style.display = "block";
    selectedBox.style.left = "".concat(rect.left - 2, "px");
    selectedBox.style.top = "".concat(rect.top - 2, "px");
    selectedBox.style.width = "".concat(rect.width + 4, "px");
    selectedBox.style.height = "".concat(rect.height + 4, "px");

    // 更新选中标签位置和内容
    selectedLabel.style.display = "block";
    selectedLabel.textContent = "\u2713 <".concat(element.tagName.toLowerCase(), ">");
```

```
    // 计算标签位置，确保不超出视窗
    var labelTop = rect.top - 28;
    var labelLeft = rect.left;

    // 如果标签会超出顶部，显示在元素下方
    if (labelTop < 5) {
      labelTop = rect.bottom + 5;
    }

    // 如果标签会超出右侧，向左调整
    var labelWidth = selectedLabel.offsetWidth || 100; // 预估宽度
    if (labelLeft + labelWidth > window.innerWidth - 10) {
      labelLeft = window.innerWidth - labelWidth - 10;
    }
    selectedLabel.style.left = "".concat(Math.max(5, labelLeft), "px");
    selectedLabel.style.top = "".concat(labelTop, "px");
    console.log("Selected highlight positioned at:", {
      left: selectedBox.style.left,
      top: selectedBox.style.top,
      width: selectedBox.style.width,
      height: selectedBox.style.height
    });
  }
  function getElementSelector(element) {
    if (!(element instanceof Element)) throw new Error('Argument must be a DOM element');
    var segments = [];
    var current = element;
    while (current !== document.documentElement) {
      var selector = '';
      // 优先检查唯一ID
      if (current.id && document.querySelectorAll("#".concat(current.id)).length === 1) {
        segments.unshift("#".concat(current.id));
        break; // ID唯一，无需继续向上
      }

      // 生成类名选择器（取第一个有效类名）
      var classes = Array.from(current.classList).filter(function (c) {
        return !c.startsWith('js-');
      });
      var className = classes.length > 0 ? ".".concat(classes[0]) : '';

      // 生成位置索引（nth-child）
      var tag = current.tagName.toLowerCase();
      if (!className) {
        var siblings = Array.from(current.parentNode.children);
        var index = siblings.findIndex(function (el) {
          return el === current;
        }) + 1;
        selector = "".concat(tag, ":nth-child(").concat(index, ")");
      } else {
        selector = className;
      }
      segments.unshift(selector);
      current = current.parentElement;
    }

    // 处理根元素
    if (current === document.documentElement) {
      segments.unshift('html');
    }
    return segments.join(' > ');
  }

  // 获取元素文本内容
  function getElementText(element) {
    var _element$textContent;
    if (element.tagName === "INPUT") {
      return element.value || element.placeholder || "";
    }
    if (element.tagName === "TEXTAREA") {
      return element.value || element.placeholder || "";
    }
    var text = ((_element$textContent = element.textContent) === null || _element$textContent === void 0 ? void 0 : _element$textCont
    return text.length > 50 ? text.substring(0, 50) + "..." : text;
  }

  // 获取元素属性信息
  function getElementAttributes(element) {
    var attrs = {};
    for (var i = 0; i < element.attributes.length; i++) {
      var attr = element.attributes[i];
      attrs[attr.name] = attr.value;
    }
    return attrs;
  }

  // 鼠标悬停事件处理
  function handleMouseOver(e) {
    if (!highlightEnabled) return;
    var target = e.target;
    if (!target || target === overlay || target === highlightBox || target === tagLabel || target === selectedBox || target === selec
      return;
    }

    // 避免高亮 html 和 body 元素
    if (target === document.documentElement || target === document.body) {
      return;
    }

    // 如果是已选中的元素，不显示悬停高亮
    if (target === selectedElement) {
```

```
      highlightBox.style.display = "none";
      tagLabel.style.display = "none";
      return;
    }
    var rect = target.getBoundingClientRect();
    var selector = getElementSelector(target);
    var text = getElementText(target);
    var attributes = getElementAttributes(target);

    // 更新悬停高亮框位置
    highlightBox.style.display = "block";
    highlightBox.style.left = "".concat(rect.left - 2, "px");
    highlightBox.style.top = "".concat(rect.top - 2, "px");
    highlightBox.style.width = "".concat(rect.width + 4, "px");
    highlightBox.style.height = "".concat(rect.height + 4, "px");

    // 更新标签位置和内容
    tagLabel.style.display = "block";
    tagLabel.textContent = "<".concat(target.tagName.toLowerCase(), ">");

    // 计算标签位置，确保不超出视窗
    var labelTop = rect.top - 22;
    var labelLeft = rect.left;

    // 如果标签会超出顶部，显示在元素下方
    if (labelTop < 0) {
      labelTop = rect.bottom + 5;
    }

    // 如果标签会超出右侧，向左调整
    if (labelLeft + tagLabel.offsetWidth > window.innerWidth) {
      labelLeft = window.innerWidth - tagLabel.offsetWidth - 5;
    }
    tagLabel.style.left = "".concat(Math.max(0, labelLeft), "px");
    tagLabel.style.top = "".concat(labelTop, "px");

    // 发送消息到父窗口
    var elementInfo = {
      tagName: target.tagName.toLowerCase(),
      rect: {
        left: rect.left,
        top: rect.top,
        right: rect.right,
        bottom: rect.bottom,
        width: rect.width,
        height: rect.height,
        x: rect.x,
        y: rect.y
      },
      selector: selector,
      text: text,
      attributes: attributes,
      url: window.location.href,
      path: window.location.pathname,
      timestamp: Date.now()
    };
    try {
      window.parent.postMessage({
        type: "iframe-element-hover",
        data: elementInfo,
        source: "iframe-highlight-injector"
      }, "*");
    } catch (error) {
      console.warn("无法发送消息到父窗口:", error);
    }
  }

  // 鼠标离开事件处理
  function handleMouseOut(e) {
    if (!highlightEnabled) return;
    var relatedTarget = e.relatedTarget;

    // 如果鼠标移动到高亮相关元素上，不隐藏高亮
    if (relatedTarget && (relatedTarget === highlightBox || relatedTarget === tagLabel || relatedTarget === overlay || relatedTarget
      return;
    }
    highlightBox.style.display = "none";
    tagLabel.style.display = "none";
    try {
      window.parent.postMessage({
        type: "iframe-element-hover",
        data: null,
        source: "iframe-highlight-injector"
      }, "*");
    } catch (error) {
      console.warn("无法发送消息到父窗口:", error);
    }
  }

  // 点击事件处理
  function handleClick(e) {
    var target = e.target;
    if (!target || target === overlay || target === highlightBox || target === tagLabel || target === selectedBox || target === selec
      return;
    }

    // 避免处理 html 和 body 元素
    if (target === document.documentElement || target === document.body) {
      return;
    }

    // 检查是否是交互元素，这些元素需要保留默认行为
```

```
    var isInteractiveElement = ['input', 'textarea', 'select', 'button', 'a'].includes(target.tagName.toLowerCase());

    // 如果高亮功能启用，对于非交互元素阻止默认行为和事件传播
    if (highlightEnabled) {
      e.preventDefault();
      e.stopPropagation();
    }
    var rect = target.getBoundingClientRect();
    var selector = getElementSelector(target);
    var text = getElementText(target);
    var attributes = getElementAttributes(target);
    console.log("Element clicked:", {
      tagName: target.tagName,
      selector: selector,
      rect: rect
    });

    // 立即更新选中高亮
    updateSelectedHighlight(target);

    // 隐藏悬停高亮，因为现在是选中状态
    highlightBox.style.display = "none";
    tagLabel.style.display = "none";
    var elementInfo = {
      tagName: target.tagName.toLowerCase(),
      rect: {
        left: rect.left,
        top: rect.top,
        right: rect.right,
        bottom: rect.bottom,
        width: rect.width,
        height: rect.height,
        x: rect.x,
        y: rect.y
      },
      selector: selector,
      text: text,
      attributes: attributes,
      url: window.location.href,
      path: window.location.pathname,
      timestamp: Date.now()
    };
    try {
      window.parent.postMessage({
        type: "iframe-element-click",
        data: elementInfo,
        source: "iframe-highlight-injector"
      }, "*");
    } catch (error) {
      console.warn("无法发送消息到父窗口:", error);
    }
  }

  // 监听来自父窗口的消息
  function handleParentMessage(event) {
    console.log("Received message from parent:", event.data);
    if (event.data.type === "iframe-highlight-toggle") {
      var enabled = event.data.enabled;
      console.log("Highlight toggle:", enabled);
      if (enabled) {
        enableHighlight();
      } else {
        disableHighlight();
      }
    } else if (event.data.type === "enable-iframe-highlight") {
      console.log("Enable iframe highlight");
      enableHighlight();
    } else if (event.data.type === "disable-iframe-highlight") {
      console.log("Disable iframe highlight");
      disableHighlight();
    } else if (event.data.type === "toggle-iframe-highlight") {
      var _enabled = event.data.enabled !== undefined ? event.data.enabled : !highlightEnabled;
      console.log("Toggle iframe highlight to:", _enabled);
      if (_enabled) {
        enableHighlight();
      } else {
        disableHighlight();
      }
    } else if (event.data.type === "update-selected-element") {
      var selector = event.data.selector;
      console.log("Update selected element with selector:", selector);
      if (selector) {
        try {
          var element = document.querySelector(selector);
          console.log("Found element by selector:", element);
          updateSelectedHighlight(element);
        } catch (error) {
          console.warn("Failed to select element:", error);
          updateSelectedHighlight(null);
        }
      } else {
        updateSelectedHighlight(null);
      }
    } else if (event.data.type === "clear-selected-element") {
      console.log("Clear selected element");
      updateSelectedHighlight(null);
    }
  }

  // 启用高亮功能
  function enableHighlight() {
    console.log("Enabling highlight");
    document.addEventListener("mouseover", handleMouseOver, true);
```

```
    document.addEventListener("mouseout", handleMouseOut, true);
    document.addEventListener("click", handleClick, true);
    highlightEnabled = true;
    overlay.style.display = "block";
  }

  // 禁用高亮功能
  function disableHighlight() {
    console.log("Disabling highlight");
    highlightEnabled = false;
    // 保持事件监听器，但通过 highlightEnabled 变量控制行为
    // 这样可以保留选中状态的显示
    highlightBox.style.display = "none";
    tagLabel.style.display = "none";
    // 不隐藏 selectedBox 和 selectedLabel，保留选中状态
  }

  // 完全禁用高亮功能（移除所有监听器）
  function fullyDisableHighlight() {
    console.log("Fully disabling highlight");
    highlightEnabled = false;
    document.removeEventListener("mouseover", handleMouseOver, true);
    document.removeEventListener("mouseout", handleMouseOut, true);
    document.removeEventListener("click", handleClick, true);
    overlay.style.display = "none";
    highlightBox.style.display = "none";
    tagLabel.style.display = "none";
    selectedBox.style.display = "none";
    selectedLabel.style.display = "none";
  }

  // 添加事件监听
  enableHighlight();
  window.addEventListener("message", handleParentMessage);

  // 暴露全局函数供外部调用
  window.__iframeHighlightControl = {
    enable: enableHighlight,
    disable: disableHighlight,
    fullyDisable: fullyDisableHighlight,
    isEnabled: function isEnabled() {
      return highlightEnabled;
    },
    getSelectedElement: function getSelectedElement() {
      return selectedElement;
    },
    updateSelected: updateSelectedHighlight,
    // 通过消息发送开关控制
    sendToggleMessage: function sendToggleMessage(enabled) {
      window.parent.postMessage({
        type: 'iframe-highlight-status',
        enabled: enabled || highlightEnabled,
        source: 'iframe-highlight-injector'
      }, '*');
    }
  };

  // 通知父窗口脚本已加载
  try {
    window.parent.postMessage({
      type: "iframe-highlight-ready",
      data: {
        url: window.location.href,
        userAgent: navigator.userAgent,
        timestamp: Date.now()
      },
      source: "iframe-highlight-injector"
    }, "*");
  } catch (error) {
    console.warn("无法发送就绪消息到父窗口:", error);
  }

  // 清理函数
  window.__iframeHighlightCleanup = function () {
    fullyDisableHighlight();
    window.removeEventListener("message", handleParentMessage);
    if (overlay.parentElement) {
      overlay.parentElement.removeChild(overlay);
    }
    delete window.__iframeHighlightInitialized;
    delete window.__iframeHighlightCleanup;
  };
})();

</script>

<script>
/**
 * Iframe 元素高亮注入脚本
 * 需要在目标网站中引入此脚本来支持跨域 iframe 高亮功能
 *
 * 使用方法:
 * 1. 将此脚本添加到目标网站的 HTML 中
 * 2. 或通过浏览器扩展、用户脚本等方式注入
 */

(function () {
  "use strict";

  // 检查是否在 iframe 中
  if (window.self === window.top) {
```

```
    return; // 不在 iframe 中，不执行
  }

  // 检查是否已经初始化过
  if (window.__iframeHighlightInitialized) {
    return;
  }
  window.__iframeHighlightInitialized = true;
  console.log("Iframe 高亮脚本已加载");

  // 创建高亮覆盖层
  var overlay = document.createElement("div");
  overlay.id = "iframe-highlight-overlay";
  overlay.style.cssText = "\n    position: fixed;\n    top: 0;\n    left: 0;\n    width: 100vw;\n    height: 100vh;\n    pointer-even

  // 创建悬停高亮框（虚线边框）
  var highlightBox = document.createElement("div");
  highlightBox.id = "iframe-highlight-box";
  highlightBox.style.cssText = "\n    position: absolute;\n    border: 2px dashed #007AFF;\n    background: rgba(0, 122, 255, 0.08);\

  // 创建选中节点的常驻高亮框（实线边框）
  var selectedBox = document.createElement("div");
  selectedBox.id = "iframe-selected-box";
  selectedBox.style.cssText = "\n    position: absolute;\n    border: 2px solid #007AFF;\n    pointer-events: none;\n    display: non

  // 创建悬停标签显示
  var tagLabel = document.createElement("div");
  tagLabel.id = "iframe-tag-label";
  tagLabel.style.cssText = "\n    position: absolute;\n    background: #007AFF;\n    color: white;\n    padding: 2px 6px;\n    font-s

  // 创建选中节点标签
  var selectedLabel = document.createElement("div");
  selectedLabel.id = "iframe-selected-label";
  selectedLabel.style.cssText = "\n    position: absolute;\n    background: #007AFF;\n    color: white;\n    padding: 3px 8px;\n    f
  overlay.appendChild(highlightBox);
  overlay.appendChild(selectedBox);
  overlay.appendChild(tagLabel);
  overlay.appendChild(selectedLabel);
  document.body.appendChild(overlay);

  // 存储当前选中的元素
  var selectedElement = null;
  var highlightEnabled = false;

  // 更新选中元素的高亮显示
  function updateSelectedHighlight(element) {
    console.log("updateSelectedHighlight called with:", element);
    if (!element) {
      selectedBox.style.display = "none";
      selectedLabel.style.display = "none";
      selectedElement = null;
      console.log("Cleared selected highlight");
      return;
    }
    selectedElement = element;
    var rect = element.getBoundingClientRect();
    console.log("Selected element rect:", rect);

    // 更新选中高亮框位置
    selectedBox.style.display = "block";
    selectedBox.style.left = "".concat(rect.left - 2, "px");
    selectedBox.style.top = "".concat(rect.top - 2, "px");
    selectedBox.style.width = "".concat(rect.width + 4, "px");
    selectedBox.style.height = "".concat(rect.height + 4, "px");

    // 更新选中标签位置和内容
    selectedLabel.style.display = "block";
    selectedLabel.textContent = "\u2713 <".concat(element.tagName.toLowerCase(), ">");

    // 计算标签位置，确保不超出视窗
    var labelTop = rect.top - 28;
    var labelLeft = rect.left;

    // 如果标签会超出顶部，显示在元素下方
    if (labelTop < 5) {
      labelTop = rect.bottom + 5;
    }

    // 如果标签会超出右侧，向左调整
    var labelWidth = selectedLabel.offsetWidth || 100; // 预估宽度
    if (labelLeft + labelWidth > window.innerWidth - 10) {
      labelLeft = window.innerWidth - labelWidth - 10;
    }
    selectedLabel.style.left = "".concat(Math.max(5, labelLeft), "px");
    selectedLabel.style.top = "".concat(labelTop, "px");
    console.log("Selected highlight positioned at:", {
      left: selectedBox.style.left,
      top: selectedBox.style.top,
      width: selectedBox.style.width,
      height: selectedBox.style.height
    });
  }
  function getElementSelector(element) {
    if (!(element instanceof Element)) throw new Error('Argument must be a DOM element');
    var segments = [];
    var current = element;
    while (current !== document.documentElement) {
      var selector = '';
      // 优先检查唯一ID
      if (current.id && document.querySelectorAll("#".concat(current.id)).length === 1) {
        segments.unshift("#".concat(current.id));
```

```
        break; // ID唯一，无需继续向上
      }

      // 生成类名选择器（取第一个有效类名）
      var classes = Array.from(current.classList).filter(function (c) {
        return !c.startsWith('js-');
      });
      var className = classes.length > 0 ? ".".concat(classes[0]) : '';

      // 生成位置索引（nth-child）
      var tag = current.tagName.toLowerCase();
      if (!className) {
        var siblings = Array.from(current.parentNode.children);
        var index = siblings.findIndex(function (el) {
          return el === current;
        }) + 1;
        selector = "".concat(tag, ":nth-child(").concat(index, ")");
      } else {
        selector = className;
      }
      segments.unshift(selector);
      current = current.parentElement;
    }

    // 处理根元素
    if (current === document.documentElement) {
      segments.unshift('html');
    }
    return segments.join(' > ');
  }

  // 获取元素文本内容
  function getElementText(element) {
    var _element$textContent;
    if (element.tagName === "INPUT") {
      return element.value || element.placeholder || "";
    }
    if (element.tagName === "TEXTAREA") {
      return element.value || element.placeholder || "";
    }
    var text = ((_element$textContent = element.textContent) === null || _element$textContent === void 0 ? void 0 : _element$textCont
    return text.length > 50 ? text.substring(0, 50) + "..." : text;
  }

  // 获取元素属性信息
  function getElementAttributes(element) {
    var attrs = {};
    for (var i = 0; i < element.attributes.length; i++) {
      var attr = element.attributes[i];
      attrs[attr.name] = attr.value;
    }
    return attrs;
  }

  // 鼠标悬停事件处理
  function handleMouseOver(e) {
    if (!highlightEnabled) return;
    var target = e.target;
    if (!target || target === overlay || target === highlightBox || target === tagLabel || target === selectedBox || target === selec
      return;
    }

    // 避免高亮 html 和 body 元素
    if (target === document.documentElement || target === document.body) {
      return;
    }

    // 如果是已选中的元素，不显示悬停高亮
    if (target === selectedElement) {
      highlightBox.style.display = "none";
      tagLabel.style.display = "none";
      return;
    }
    var rect = target.getBoundingClientRect();
    var selector = getElementSelector(target);
    var text = getElementText(target);
    var attributes = getElementAttributes(target);

    // 更新悬停高亮框位置
    highlightBox.style.display = "block";
    highlightBox.style.left = "".concat(rect.left - 2, "px");
    highlightBox.style.top = "".concat(rect.top - 2, "px");
    highlightBox.style.width = "".concat(rect.width + 4, "px");
    highlightBox.style.height = "".concat(rect.height + 4, "px");

    // 更新标签位置和内容
    tagLabel.style.display = "block";
    tagLabel.textContent = "<".concat(target.tagName.toLowerCase(), ">");

    // 计算标签位置，确保不超出视窗
    var labelTop = rect.top - 22;
    var labelLeft = rect.left;

    // 如果标签会超出顶部，显示在元素下方
    if (labelTop < 0) {
      labelTop = rect.bottom + 5;
    }

    // 如果标签会超出右侧，向左调整
    if (labelLeft + tagLabel.offsetWidth > window.innerWidth) {
      labelLeft = window.innerWidth - tagLabel.offsetWidth - 5;
    }
```

```
    tagLabel.style.left = "".concat(Math.max(0, labelLeft), "px");
    tagLabel.style.top = "".concat(labelTop, "px");

    // 发送消息到父窗口
    var elementInfo = {
      tagName: target.tagName.toLowerCase(),
      rect: {
        left: rect.left,
        top: rect.top,
        right: rect.right,
        bottom: rect.bottom,
        width: rect.width,
        height: rect.height,
        x: rect.x,
        y: rect.y
      },
      selector: selector,
      text: text,
      attributes: attributes,
      url: window.location.href,
      path: window.location.pathname,
      timestamp: Date.now()
    };
    try {
      window.parent.postMessage({
        type: "iframe-element-hover",
        data: elementInfo,
        source: "iframe-highlight-injector"
      }, "*");
    } catch (error) {
      console.warn("无法发送消息到父窗口:", error);
    }
  }

  // 鼠标离开事件处理
  function handleMouseOut(e) {
    if (!highlightEnabled) return;
    var relatedTarget = e.relatedTarget;

    // 如果鼠标移动到高亮相关元素上，不隐藏高亮
    if (relatedTarget && (relatedTarget === highlightBox || relatedTarget === tagLabel || relatedTarget === overlay || relatedTarget =
      return;
    }
    highlightBox.style.display = "none";
    tagLabel.style.display = "none";
    try {
      window.parent.postMessage({
        type: "iframe-element-hover",
        data: null,
        source: "iframe-highlight-injector"
      }, "*");
    } catch (error) {
      console.warn("无法发送消息到父窗口:", error);
    }
  }

  // 点击事件处理
  function handleClick(e) {
    var target = e.target;
    if (!target || target === overlay || target === highlightBox || target === tagLabel || target === selectedBox || target === selec
      return;
    }

    // 避免处理 html 和 body 元素
    if (target === document.documentElement || target === document.body) {
      return;
    }

    // 检查是否是交互元素，这些元素需要保留默认行为
    var isInteractiveElement = ['input', 'textarea', 'select', 'button', 'a'].includes(target.tagName.toLowerCase());

    // 如果高亮功能启用，对于非交互元素阻止默认行为和事件传播
    if (highlightEnabled) {
      e.preventDefault();
      e.stopPropagation();
    }
    var rect = target.getBoundingClientRect();
    var selector = getElementSelector(target);
    var text = getElementText(target);
    var attributes = getElementAttributes(target);
    console.log("Element clicked:", {
      tagName: target.tagName,
      selector: selector,
      rect: rect
    });

    // 立即更新选中高亮
    updateSelectedHighlight(target);

    // 隐藏悬停高亮，因为现在是选中状态
    highlightBox.style.display = "none";
    tagLabel.style.display = "none";
    var elementInfo = {
      tagName: target.tagName.toLowerCase(),
      rect: {
        left: rect.left,
        top: rect.top,
        right: rect.right,
        bottom: rect.bottom,
        width: rect.width,
        height: rect.height,
        x: rect.x,
```

```
      y: rect.y
    },
    selector: selector,
    text: text,
    attributes: attributes,
    url: window.location.href,
    path: window.location.pathname,
    timestamp: Date.now()
  };
  try {
    window.parent.postMessage({
      type: "iframe-element-click",
      data: elementInfo,
      source: "iframe-highlight-injector"
    }, "*");
  } catch (error) {
    console.warn("无法发送消息到父窗口:", error);
  }
}

// 监听来自父窗口的消息
function handleParentMessage(event) {
  console.log("Received message from parent:", event.data);
  if (event.data.type === "iframe-highlight-toggle") {
    var enabled = event.data.enabled;
    console.log("Highlight toggle:", enabled);
    if (enabled) {
      enableHighlight();
    } else {
      disableHighlight();
    }
  } else if (event.data.type === "enable-iframe-highlight") {
    console.log("Enable iframe highlight");
    enableHighlight();
  } else if (event.data.type === "disable-iframe-highlight") {
    console.log("Disable iframe highlight");
    disableHighlight();
  } else if (event.data.type === "toggle-iframe-highlight") {
    var _enabled = event.data.enabled !== undefined ? event.data.enabled : !highlightEnabled;
    console.log("Toggle iframe highlight to:", _enabled);
    if (_enabled) {
      enableHighlight();
    } else {
      disableHighlight();
    }
  } else if (event.data.type === "update-selected-element") {
    var selector = event.data.selector;
    console.log("Update selected element with selector:", selector);
    if (selector) {
      try {
        var element = document.querySelector(selector);
        console.log("Found element by selector:", element);
        updateSelectedHighlight(element);
      } catch (error) {
        console.warn("Failed to select element:", error);
        updateSelectedHighlight(null);
      }
    } else {
      updateSelectedHighlight(null);
    }
  } else if (event.data.type === "clear-selected-element") {
    console.log("Clear selected element");
    updateSelectedHighlight(null);
  }
}

// 启用高亮功能
function enableHighlight() {
  console.log("Enabling highlight");
  document.addEventListener("mouseover", handleMouseOver, true);
  document.addEventListener("mouseout", handleMouseOut, true);
  document.addEventListener("click", handleClick, true);
  highlightEnabled = true;
  overlay.style.display = "block";
}

// 禁用高亮功能
function disableHighlight() {
  console.log("Disabling highlight");
  highlightEnabled = false;
  // 保持事件监听器，但通过 highlightEnabled 变量控制行为
  // 这样可以保留选中状态的显示
  highlightBox.style.display = "none";
  tagLabel.style.display = "none";
  // 不隐藏 selectedBox 和 selectedLabel，保留选中状态
}

// 完全禁用高亮功能（移除所有监听器）
function fullyDisableHighlight() {
  console.log("Fully disabling highlight");
  highlightEnabled = false;
  document.removeEventListener("mouseover", handleMouseOver, true);
  document.removeEventListener("mouseout", handleMouseOut, true);
  document.removeEventListener("click", handleClick, true);
  overlay.style.display = "none";
  highlightBox.style.display = "none";
  tagLabel.style.display = "none";
  selectedBox.style.display = "none";
  selectedLabel.style.display = "none";
}

// 添加事件监听
enableHighlight();
```

```
  window.addEventListener("message", handleParentMessage);

  // 暴露全局函数供外部调用
  window.__iframeHighlightControl = {
    enable: enableHighlight,
    disable: disableHighlight,
    fullyDisable: fullyDisableHighlight,
    isEnabled: function isEnabled() {
      return highlightEnabled;
    },
    getSelectedElement: function getSelectedElement() {
      return selectedElement;
    },
    updateSelected: updateSelectedHighlight,
    // 通过消息发送开关控制
    sendToggleMessage: function sendToggleMessage(enabled) {
      window.parent.postMessage({
        type: 'iframe-highlight-status',
        enabled: enabled || highlightEnabled,
        source: 'iframe-highlight-injector'
      }, '*');
    }
  };

  // 通知父窗口脚本已加载
  try {
    window.parent.postMessage({
      type: "iframe-highlight-ready",
      data: {
        url: window.location.href,
        userAgent: navigator.userAgent,
        timestamp: Date.now()
      },
      source: "iframe-highlight-injector"
    }, "*");
  } catch (error) {
    console.warn("无法发送就绪消息到父窗口:", error);
  }

  // 清理函数
  window.__iframeHighlightCleanup = function () {
    fullyDisableHighlight();
    window.removeEventListener("message", handleParentMessage);
    if (overlay.parentElement) {
      overlay.parentElement.removeChild(overlay);
    }
    delete window.__iframeHighlightInitialized;
    delete window.__iframeHighlightCleanup;
  };
})();

</script>
<style>
#minimax-floating-ball {
  position: fixed;
  bottom: 20px;
  right: 20px;
  padding: 10px 12px;
  background: #222222;
  border-radius: 12px;
  display: flex;
  align-items: center;
  color: #F8F8F8;
  font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto, Oxygen, Ubuntu, Cantarell, 'Open Sans', 'Helvetica Neue', sans-
  z-index: 9999;
  transition: all 0.3s ease;
  overflow: hidden;
  cursor: pointer;
}

#minimax-floating-ball:hover {
  transform: translateY(-2px);
  background: #383838;
}
.minimax-ball-content {
  display: flex;
  align-items: center;
  gap: 8px;
}

.minimax-logo-wave {

  width: 20px;
  height: 17px;
  background-image: url("data:image/svg+xml,%3Csvg xmlns='http://www.w3.org/2000/svg' width='20' height='17' viewBox='0 0 20 17' fill
  background-repeat: no-repeat;
  background-position: center;
}

.minimax-ball-text {
  font-size: 12px;
  font-weight: 500;
  white-space: nowrap;
}

.minimax-close-icon {
  margin-left: 8px;
  font-size: 16px;
  width: 18px;
  height: 18px;
  display: flex;
  align-items: center;
  justify-content: center;
```

```
  border-radius: 50%;
  cursor: pointer;
  opacity: 0.7;
  transition: opacity 0.2s ease;
}

.minimax-close-icon:hover {
  opacity: 1;
}
</style>
<div id="minimax-floating-ball">
  <div class="minimax-ball-content">
    <div class="minimax-logo-wave"></div>
    <span class="minimax-ball-text">Created by MiniMax Agent</span>
  </div>
  <div class="minimax-close-icon">×</div>
</div>
<script>
// Initialize floating ball functionality
function initFloatingBall() {
  const ball = document.getElementById('minimax-floating-ball');
  if (!ball) return;

  // Initial animation
  ball.style.opacity = '0';
  ball.style.transform = 'translateY(20px)';

  setTimeout(() => {
    ball.style.opacity = '1';
    ball.style.transform = 'translateY(0)';
  }, 500);

  // Handle logo click
  const ballContent = ball.querySelector('.minimax-ball-content');
  ballContent.addEventListener('click', function (e) {
    e.stopPropagation();
    window.open('https://agent.minimax.io/', '_blank');
    ball.style.transform = 'scale(0.95)';
    setTimeout(() => {
      ball.style.transform = 'scale(1)';
    }, 100);
  });

  // Handle close button click
  const closeIcon = ball.querySelector('.minimax-close-icon');
  closeIcon.addEventListener('click', function (e) {
    e.stopPropagation();
    ball.style.opacity = '0';
    ball.style.transform = 'translateY(20px)';

    setTimeout(() => {
      ball.style.display = 'none';
    }, 300);
  });
}

// Initialize when DOM is ready
document.addEventListener('DOMContentLoaded', initFloatingBall);
</script>
</body>
</html>
```